

Содержание

1 Собственная реализация Binary Heap	2
1.0.1 Общая архитектура	2
1.0.2 Свойство кучи	2
1.0.3 Операция push	3
1.0.4 Операция pop	3
1.0.5 Операция heapify	3
1.0.6 Поддерживаемые операции	4
1.1 Тестирование реализации	5
2 Сравнимые реализации	6
3 Benchmark results	7
3.1 Push sequential	7
3.2 Push reverse sequential	9
3.3 Push random	11
3.4 Heapify random	13
3.5 Pop all after heapify	15
3.6 Alternating push/pop	17
3.7 Top-k largest	19
3.8 Общий вывод по benchmark-ам Binary Heap	21

Собственная реализация Binary Heap

В рамках работы была реализована собственная структура данных типа binary heap как на Python, так и на C++. Обе реализации построены на одной и той же основной идее: куча хранится в массиве, а порядок элементов поддерживается с помощью операций `sift up` и `sift down`.

В данной работе используется **min-heap**: минимальный элемент всегда находится в корне кучи. Поэтому операция получения минимального элемента выполняется за $O(1)$, а операции вставки и удаления корня в худшем случае требуют восстановления свойства кучи за $O(\log N)$.

Общая архитектура

Binary heap хранится не как дерево из отдельных узлов, а как **обычный массив**. Логическая древовидная структура задается индексами элементов.

Для элемента с индексом `i` используются следующие формулы:

note

```
1 parent(i)      = (i - 1) / 2
2 left_child(i)  = 2 * i + 1
3 right_child(i) = 2 * i + 2
```

За счет этого не требуется хранить указатели на детей и родителя. Все связи между вершинами вычисляются по индексам. Такая организация делает реализацию компактной и хорошо подходящей для хранения в непрерывном массиве.

Свойство кучи

Для min-heap выполняется следующее условие:

note

```
1 parent <= left_child
2 parent <= right_child
```

То есть каждый родитель не больше своих детей. Из этого следует, что минимальный элемент всей структуры всегда находится в корне, то есть в позиции `0`.

При этом куча не является полностью отсортированным массивом. Она гарантирует только локальный порядок между родителем и детьми. Поэтому поиск произвольного элемента в куче не является эффективной операцией, зато быстро поддерживаются операции приоритетной очереди: вставка, получение минимума и удаление минимума.

Операция push

Операция `push` добавляет новый элемент в конец массива. После этого может нарушиться свойство кучи, потому что новый элемент может оказаться меньше своего родителя.

Для восстановления свойства кучи используется операция `sift up` :

1. новый элемент добавляется в конец массива;
2. сравнивается с родителем;
3. если новый элемент меньше родителя, они меняются местами;
4. процесс повторяется, пока элемент не окажется на корректной позиции.

В худшем случае элемент поднимается от листа до корня. Высота бинарной кучи равна $O(\log N)$, поэтому операция `push` имеет сложность $O(\log N)$. Однако в некоторых сценариях, например при вставке уже возрастающей последовательности в `min-heap`, новый элемент часто сразу оказывается на правильном месте, и фактическая стоимость операции становится близкой к $O(1)$.

Операция pop

Операция `pop` удаляет корневой элемент, то есть минимальное значение. После удаления корня нужно сохранить полную форму бинарного дерева, поэтому на место корня переносится последний элемент массива.

После этого свойство кучи может нарушиться уже сверху, поэтому используется операция `sift down` :

1. корень заменяется последним элементом;
2. последний элемент удаляется из массива;
3. новый корень сравнивается с детьми;
4. если один из детей меньше, элемент меняется местами с меньшим ребенком;
5. процесс продолжается вниз до восстановления свойства кучи.

В худшем случае элемент опускается от корня до листа, поэтому `pop` имеет сложность $O(\log N)$.

Операция heapify

Операция `heapify` строит кучу из уже существующего массива. В реализации используется классический `bottom-up` подход: обработка начинается с последнего внутреннего узла и идет к корню, для каждого узла выполняется `sift down` .

Важно, что построение кучи через `heapify` работает за $O(N)$, а не за $O(N \log N)$. Это связано с тем, что большинство вершин находятся близко к листьям и требуют очень короткого `sift down` . Поэтому суммарная стоимость всех просеиваний оказывается линейной.

Поддерживаемые операции

Реализация поддерживает основной набор операций:

- `push` — вставка элемента;
- `pop` — удаление и возврат минимального элемента;
- `peek` — получение минимального элемента без удаления;
- `heapify` — построение кучи из массива;
- `copy` — копирование структуры для корректного проведения benchmark-ов.

Таким образом, собственная реализация соответствует классической реализации бинарной кучи на массиве. Она проста, понятна и позволяет отдельно исследовать влияние порядка входных данных, стоимости `sift up`, стоимости `sift down` и различий между пользовательской и библиотечной реализацией.

Тестирование реализации

Для проверки корректности реализации `binary heap` был разработан набор тестов как для Python, так и для C++ версии.

Тесты покрывают основные аспекты работы структуры:

- вставку элементов и сохранение свойства `min-heap`;
- удаление элементов в правильном порядке;
- получение минимального элемента через `peek` ;
- построение кучи из готового массива через `heapify` ;
- работу с пустой кучей;
- корректность поведения после последовательности вставок и удалений;
- копирование структуры перед измерениями.

Таким образом, тестирование проверяет не только базовую функциональность, но и ключевые особенности реализации бинарной кучи: восстановление свойства кучи при движении элемента вверх и вниз, а также корректность построения структуры из неупорядоченного массива.

Сравниваемые реализации

В качестве эталонных реализаций для сравнения были выбраны стандартный модуль Python `heapq` и стандартный контейнер C++ `std::priority_queue`.

Выбор `heapq` обусловлен тем, что это стандартная библиотечная реализация кучи в Python. Она использует обычный список как внутреннее хранилище и реализует операции над ним в стиле binary heap. Для пользователя это не отдельный класс, а набор функций, которые работают с массивом:

- `heapq.heappush` — вставка элемента;
- `heapq.heappop` — удаление минимального элемента;
- `heapq.heapify` — построение кучи из списка.

`heapq` является хорошим baseline для Python-реализации, потому что представляет стандартный способ работы с кучей в Python. Сравнение с ним показывает, насколько сильно пользовательская реализация на Python уступает стандартной библиотечной реализации по постоянным факторам, стоимости циклов, вызовов функций и внутренней оптимизации.

Для C++ в качестве аналога используется `std::priority_queue`. Это стандартный контейнер-адаптер, который предоставляет интерфейс приоритетной очереди и обычно использует массивное хранилище, например `std::vector`, вместе с heap-операциями стандартной библиотеки.

В отличие от собственной реализации, `std::priority_queue` является более универсальной библиотечной структурой. Она скрывает внутренние детали хранения и предоставляет готовый интерфейс приоритетной очереди. Поэтому сравнение с ней позволяет оценить, насколько собственная C++ реализация конкурентоспособна относительно стандартного библиотечного решения.

Таким образом, в benchmark-ах сравниваются четыре реализации:

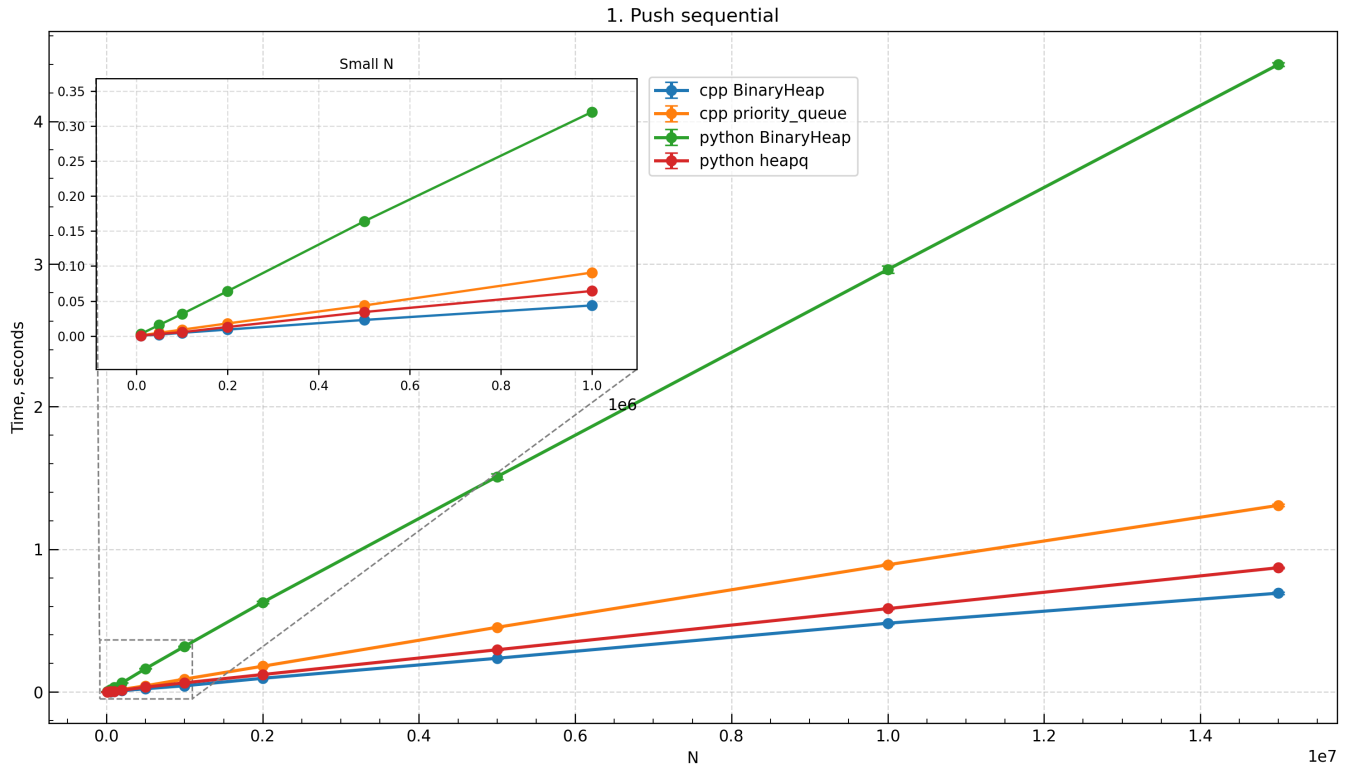
- собственная `BinaryHeap` на Python;
- стандартная Python-реализация `heapq`;
- собственная `BinaryHeap` на C++;
- стандартная C++ реализация `std::priority_queue`.

Такое сравнение позволяет отдельно увидеть влияние языка реализации, библиотечных оптимизаций, порядка входных данных и особенностей операций `push`, `pop` и `heapify`.

Benchmark results

Push sequential

В данном benchmark-case измеряется время последовательной вставки возрастающих значений в изначально пустую кучу. Для каждого значения N в структуру добавляются элементы $0, 1, 2, \dots, N - 1$. Таким образом, измеряется стоимость массового выполнения операции `push` на благоприятном порядке входных данных.



note

1. Push sequential

	N	cpp_BinaryHeap	priority_queue	python_BinaryHeap	heapq
5	1000000	~0.043 s	~0.090 s	~0.319 s	~0.066 s
6	2000000	~0.095 s	~0.178 s	~0.635 s	~0.123 s
7	5000000	~0.234 s	~0.453 s	~1.509 s	~0.295 s
8	10000000	~0.475 s	~0.893 s	~2.945 s	~0.581 s
9	15000000	~0.685 s	~1.313 s	~4.386 s	~0.868 s

По графику и таблице видно, что все реализации демонстрируют рост, близкий к линейному. Это объясняется особенностью входных данных. Для `min-heap` вставка возрастающей последовательности является почти лучшим случаем: каждый новый элемент больше или равен уже находящимся в куче элементам, поэтому после добавления в конец массива он обычно не поднимается вверх.

Формально операция `push` имеет сложность $O(\log N)$, потому что в худшем случае элемент может пройти путь от листа до корня. Однако в данном кейсе `sift up` почти сразу останавливается после сравнения с родителем. Поэтому фактическая стоимость одной вставки близка к $O(1)$, а серия из N вставок ведет себя почти как $O(N)$.

Собственная Python-реализация оказывается самой медленной. Даже в благоприятном случае каждая операция проходит через Python-метод `push`, добавление в список, вызов `_sift_up`, вычисление индекса родителя и сравнение элементов. Эти накладные расходы повторяются миллионы раз и дают заметный разрыв.

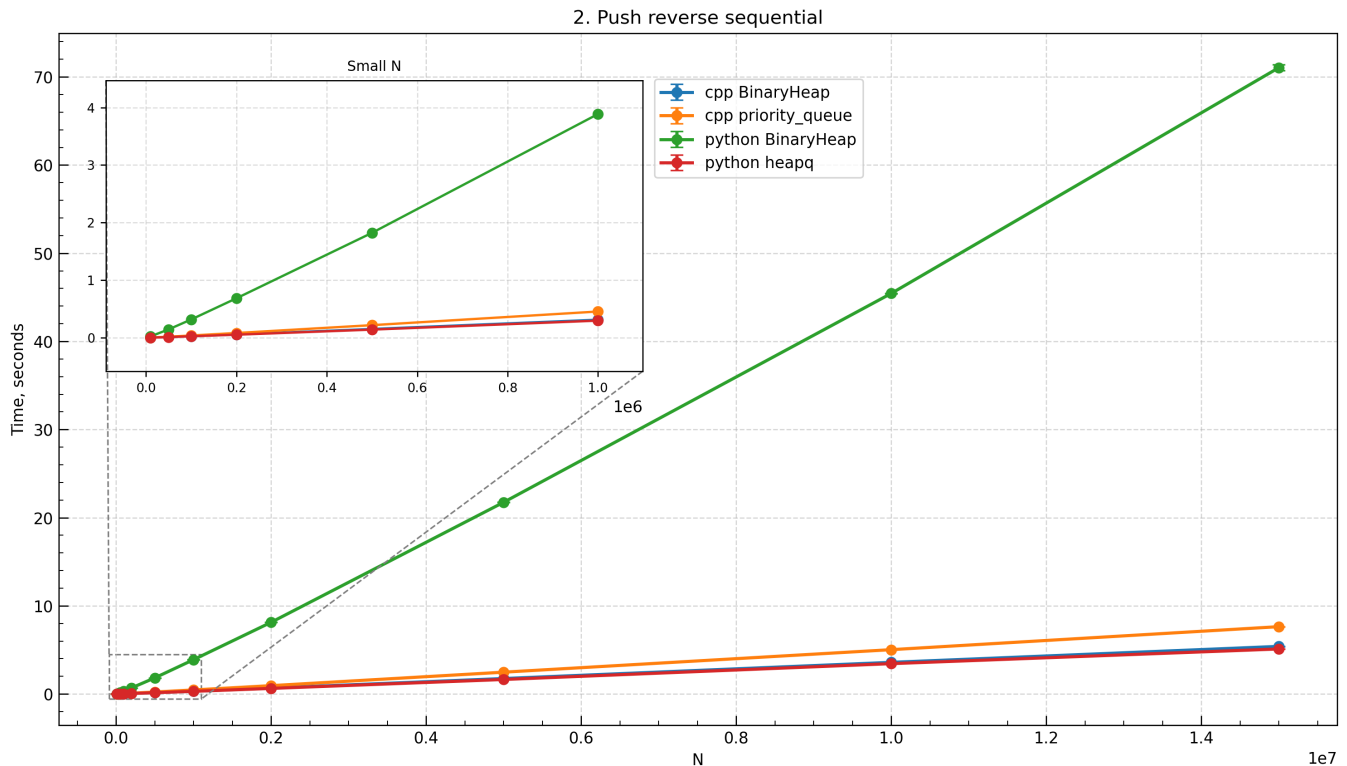
Стандартный `heapq` работает значительно быстрее собственной Python-реализации. Он использует более оптимизированную библиотечную реализацию операций над кучей и имеет меньшие постоянные расходы на одну вставку.

Среди C++-реализаций собственная `BinaryHeap` оказывается быстрее `std::priority_queue`. В данном простом сценарии собственная реализация выполняет очень прямолинейную операцию: добавить элемент в конец и почти сразу завершить `sift up`. У `std::priority_queue` может быть больший постоянный overhead как у более универсального контейнера-адаптера.

Таким образом, benchmark-case `Push sequential` показывает лучший сценарий для операции вставки в min-heap. Несмотря на теоретическую оценку $O(\log N)$, фактическое поведение близко к линейному для всей серии операций, потому что входные данные почти не требуют перемещения элементов вверх по дереву.

Push reverse sequential

В данном benchmark-case измеряется время вставки значений в обратном порядке. Для каждого значения N в изначально пустую кучу добавляются элементы $N - 1, N - 2, \dots, 1, 0$. Для min-heap такой порядок является неблагоприятным, потому что каждый следующий элемент меньше большинства уже вставленных элементов.



note

1 2. Push reverse sequential

	N	cpp_BinaryHeap	cpp_priority_queue	python_BinaryHeap	heapq
5	1000000	~0.314 s	~0.456 s	~3.890 s	~0.298 s
6	2000000	~0.672 s	~0.940 s	~8.117 s	~0.625 s
7	5000000	~1.749 s	~2.466 s	~21.714 s	~1.643 s
8	10000000	~3.575 s	~5.032 s	~45.470 s	~3.438 s
9	15000000	~5.407 s	~7.621 s	~70.544 s	~5.114 s

По графику видно, что время выполнения резко увеличивается по сравнению с `Push sequential`. Это ожидаемо: при вставке убывающей последовательности каждый новый элемент часто становится новым минимумом и вынужден подниматься почти до корня кучи.

В этом сценарии операция `sift up` проявляется почти в худшем случае. Высота бинарной кучи равна $O(\log N)$, поэтому отдельная вставка может требовать $O(\log N)$ сравнений и перестановок. Серия из N таких вставок приближается к поведению $O(N \log N)$, что и объясняет сильный рост времени.

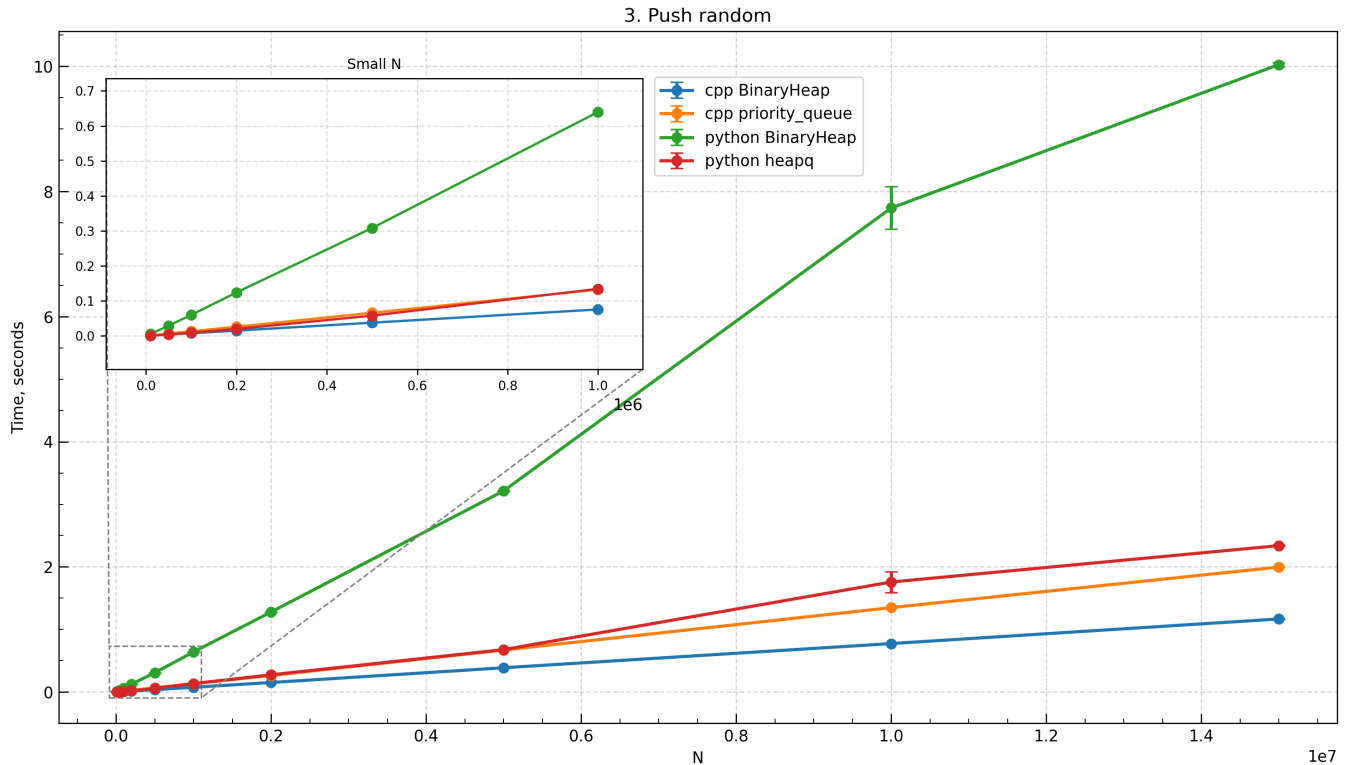
Особенно сильно деградация видна у собственной Python-реализации. Перемещение элемента вверх выполняется в цикле Python, с вычислением индексов, обращениями к списку и вызовом `_swap`. Чем больше становится куча, тем больше уровней приходится проходить, и накладные расходы Python умножаются на большое число шагов `sift up`.

`heapq` и C++-реализации оказываются значительно быстрее, потому что выполняют ту же логическую работу с меньшими постоянными расходами. При этом данный кейс показывает, что для кучи порядок входных данных может иметь решающее значение: одна и та же операция `push` может быть почти константной на возрастающей последовательности и логарифмической на убывающей.

Таким образом, benchmark-case `Push reverse sequential` демонстрирует неблагоприятный сценарий вставки в min-heap. Он хорошо показывает реальную цену операции `sift up` и объясняет, почему теоретическая оценка $O(\log N)$ для вставки важна, даже если на некоторых данных вставка работает почти за $O(1)$.

Push random

В данном benchmark-case измеряется время вставки случайных уникальных значений в изначально пустую кучу. Для каждого значения `N` заранее генерируется набор из `N` различных случайных чисел, после чего в измеряемой части выполняется последовательность операций `push`.



note

3. Push random

	N	cpp_BinaryHeap	cpp_priority_queue	python_BinaryHeap	heapq
5	1000000	~0.075 s	~0.132 s	~0.641 s	~0.133 s
6	2000000	~0.151 s	~0.264 s	~1.278 s	~0.275 s
7	5000000	~0.385 s	~0.671 s	~3.206 s	~0.676 s
8	10000000	~0.774 s	~1.341 s	~7.688 s	~1.695 s
9	15000000	~1.172 s	~1.998 s	~10.006 s	~2.345 s

По графику видно, что `Push random` находится между двумя предыдущими случаями. Он заметно медленнее, чем вставка возрастающей последовательности, но значительно быстрее, чем вставка в обратном порядке.

Это объясняется тем, что случайный новый элемент иногда оказывается меньше своего родителя и должен подниматься вверх, но обычно он не проходит путь до самого корня. Поэтому средняя длина `sift up` оказывается больше, чем в лучшем случае, но меньше, чем в худшем случае.

С точки зрения теории отдельная операция `push` по-прежнему имеет худшую сложность $O(\log N)$. Однако

на случайных данных среднее число перестановок обычно существенно меньше высоты дерева. Поэтому на графике рост выглядит ближе к линейному, хотя константа выше, чем в случае `Push sequential`.

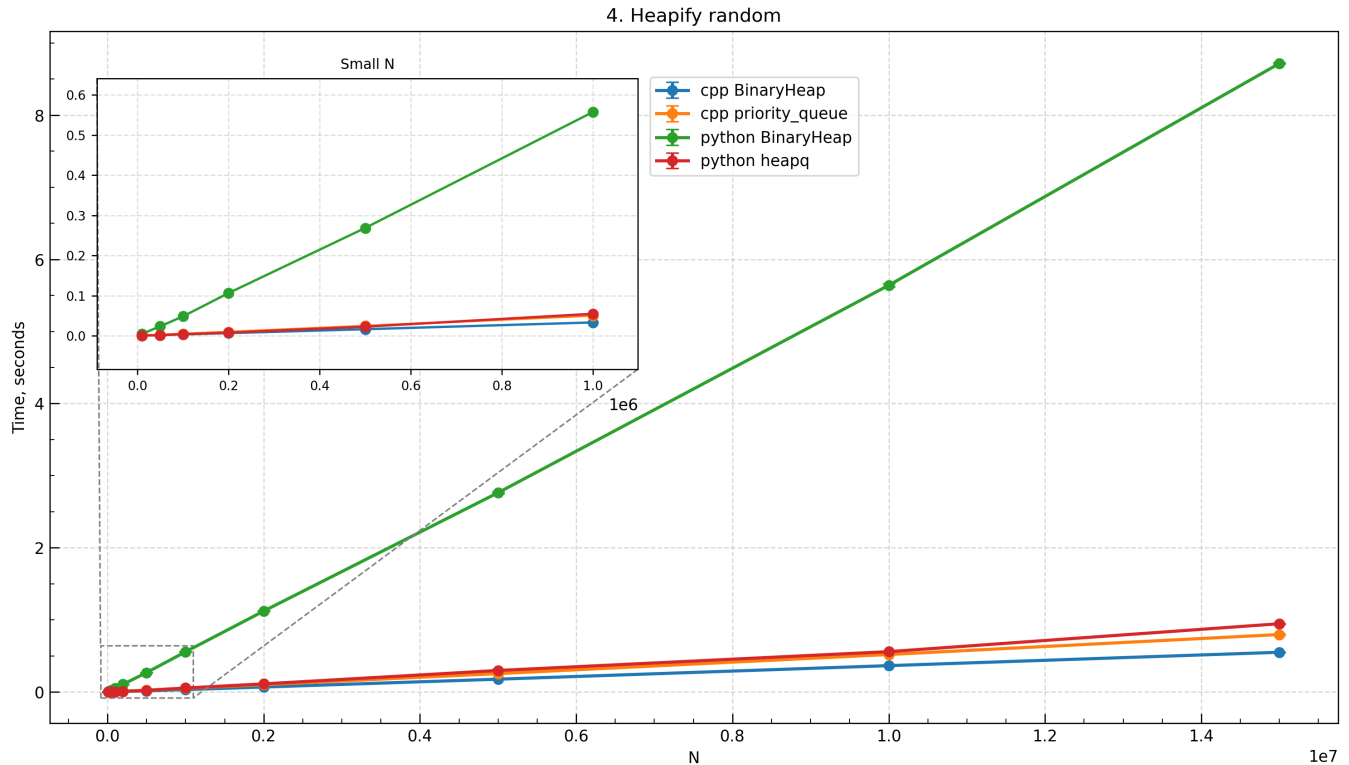
Собственная Python-реализация снова проигрывает стандартному `heapq`, потому что каждый шаг `sift up` выполняется на уровне Python-кода. Чем чаще элементу приходится подниматься, тем сильнее проявляются накладные расходы циклов, сравнений, вычисления индексов и перестановок.

Среди C++-реализаций собственная `BinaryHeap` остается быстрее `std::priority_queue`. Это связано с тем, что собственная реализация специализирована под конкретный сценарий `min-heap` и имеет более прямолинейную внутреннюю логику, тогда как `priority_queue` является универсальным контейнером-адаптером.

Таким образом, benchmark-case `Push random` показывает типичный сценарий вставки в кучу. Он подтверждает, что реальная стоимость `push` сильно зависит от того, насколько далеко новый элемент должен подниматься по дереву.

Heapify random

В данном benchmark-case измеряется время построения кучи из случайного массива. Для каждого значения `N` заранее генерируется массив из `N` случайных уникальных значений, после чего в измеряемой части выполняется операция `heapify`.



note

1 4. Heapify random

2

3	N	cpp_BinaryHeap	cpp_priority_queue	python_BinaryHeap	heapq
4					
5	1000000	~0.033 s	~0.049 s	~0.559 s	~0.054 s
6	2000000	~0.068 s	~0.101 s	~1.121 s	~0.111 s
7	5000000	~0.179 s	~0.253 s	~2.753 s	~0.286 s
8	10000000	~0.363 s	~0.520 s	~5.625 s	~0.559 s
9	15000000	~0.547 s	~0.790 s	~8.713 s	~0.941 s

По графику видно, что все реализации демонстрируют рост, близкий к линейному. Это соответствует теоретической сложности операции `heapify`: построение кучи bottom-up выполняется за $O(N)$.

Этот результат важен, потому что построение кучи через `heapify` принципиально отличается от построения через последовательные `push`. Если вставлять элементы по одному, в худшем случае можно получить $O(N \log N)$. При `heapify` структура строится снизу вверх, и большинство элементов требуют очень короткого `sift down`. Поэтому суммарная стоимость оказывается линейной.

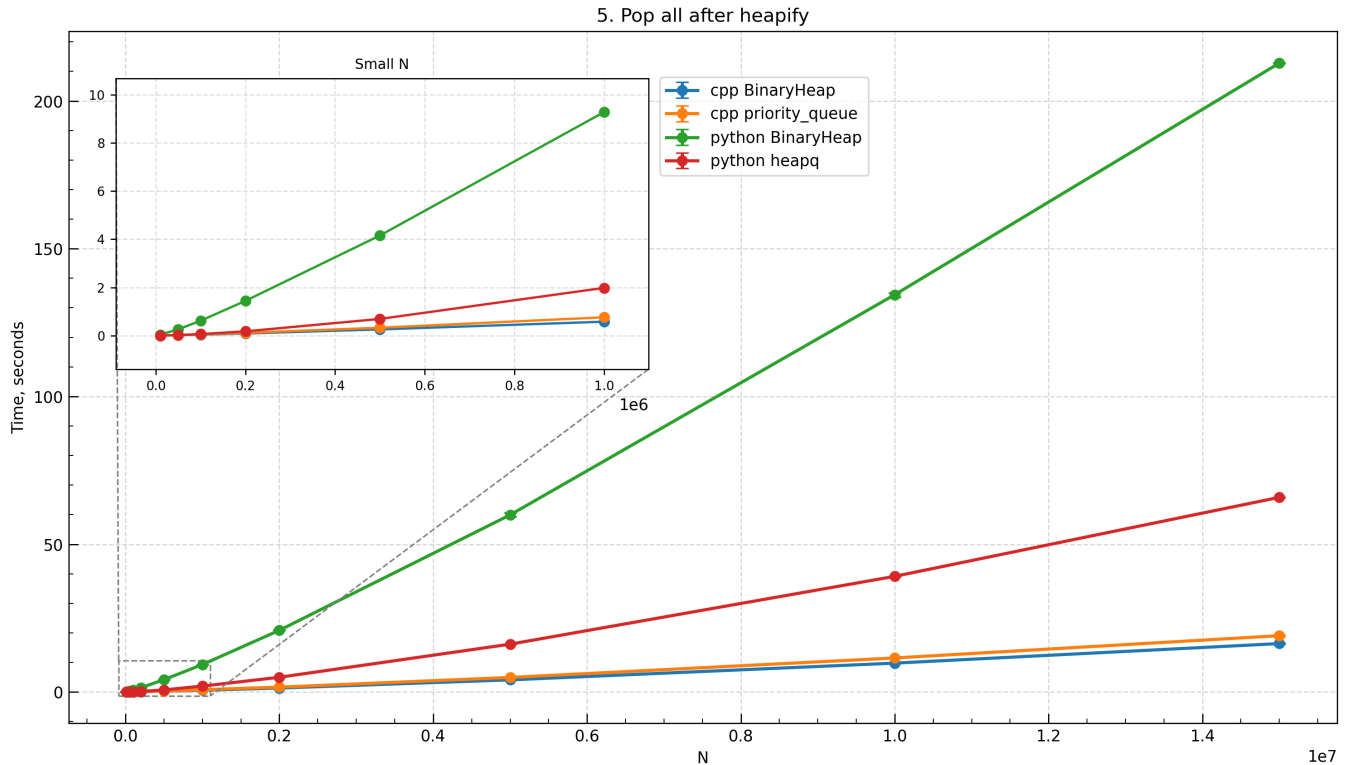
Собственная Python-реализация заметно медленнее `heapq`, хотя использует тот же алгоритмический принцип. Причина заключается в постоянных расходах: циклы, вызовы методов, вычисление индексов и перестановки выполняются на уровне Python-кода.

С++-реализации показывают наименьшее время. Собственная `BinaryHeap` оказывается быстрее `std::priority_queue`, что можно объяснить более специализированной логикой построения и меньшими накладными расходами. При этом обе реализации подтверждают линейную природу `heapify`.

Таким образом, benchmark-case `Heapify random` демонстрирует преимущество bottom-up построения кучи. Если все элементы известны заранее, `heapify` обычно эффективнее, чем последовательные вставки через `push`.

Pop all after heapify

В данном benchmark-case измеряется время последовательного удаления всех элементов из кучи после предварительного построения через `heapify`. Для каждого значения `N` сначала создается куча из случайных значений, а затем в измеряемой части выполняется `pop`, пока структура не станет пустой.



note

5. Pop all after heapify

	N	cpp_BinaryHeap	cpp_priority_queue	python_BinaryHeap	heapq
5	1000000	~0.582 s	~0.737 s	~9.266 s	~1.984 s
6	2000000	~1.321 s	~1.636 s	~21.094 s	~4.977 s
7	5000000	~4.088 s	~4.920 s	~59.261 s	~16.172 s
8	10000000	~9.750 s	~11.497 s	~135.515 s	~39.140 s
9	15000000	~16.204 s	~18.953 s	~212.792 s	~65.817 s

Этот benchmark-case является одним из самых тяжелых для бинарной кучи. Каждая операция `pop` удаляет корень, переносит последний элемент в начало массива и затем выполняет `sift down`. В худшем случае элемент проходит путь от корня до листа, то есть операция требует $O(\log N)$.

Так как удаляются все `N` элементов, суммарная сложность серии операций составляет $O(N \log N)$. Именно поэтому времена в этом кейсе значительно выше, чем в `Heapify random`: построить кучу можно за $O(N)$, но полностью разобрать ее через последовательные удаления уже стоит $O(N \log N)$.

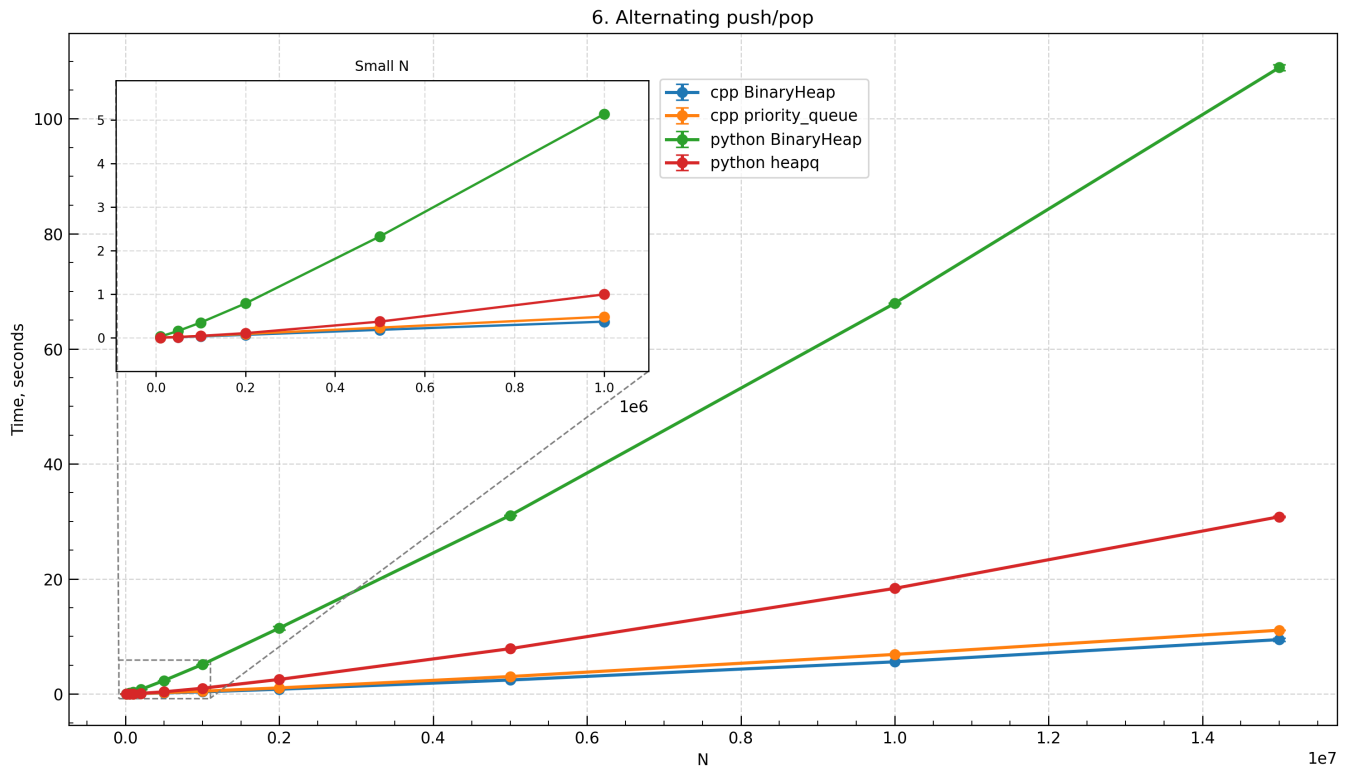
Собственная Python-реализация особенно сильно проигрывает на этом тесте. Операция `sift down` содержит цикл, несколько сравнений с детьми, выбор меньшего ребенка и перестановку элементов. При миллионах удалений эта логика выполняется огромное число раз на уровне Python-кода, поэтому накопленные накладные расходы становятся очень большими.

`heapq` также выполняет логически ту же операцию, но делает это значительно быстрее за счет стандартной оптимизированной реализации. C++-реализации оказываются самыми быстрыми, потому что вся работа выполняется как скомпилированный код с меньшими постоянными расходами.

Таким образом, benchmark-case `Pop all after heapify` показывает реальную стоимость массового удаления из кучи. Он хорошо демонстрирует отличие линейного построения `heapify` от последующего извлечения всех элементов, которое требует логарифмической работы на каждом шаге.

Alternating push/pop

В данном benchmark-case измеряется работа кучи в смешанном сценарии, где операции `push` и `pop` чередуются. Сначала структура заполняется случайными значениями примерно на половину от N , после чего выполняется последовательность из N операций, каждая из которых с некоторой вероятностью является вставкой или удалением.



note

1 6. Alternating push/pop

2

3	N	cpp_BinaryHeap	cpp_priority_queue	python_BinaryHeap	heapq
4					
5	1000000	~0.369 s	~0.480 s	~5.139 s	~0.994 s
6	2000000	~0.824 s	~1.050 s	~11.106 s	~2.519 s
7	5000000	~2.422 s	~3.027 s	~31.013 s	~7.872 s
8	10000000	~5.587 s	~6.875 s	~67.934 s	~18.331 s
9	15000000	~9.132 s	~11.089 s	~108.348 s	~30.843 s

Данный сценарий ближе к реальному использованию приоритетной очереди, чем отдельные тесты только на вставку или только на удаление. В реальных задачах куча часто не просто строится один раз, а постоянно изменяется: новые элементы добавляются, а минимальные извлекаются.

По графику видно, что время заметно выше, чем в простых тестах на `push`, но ниже, чем при полном удалении всех элементов после `heapify`. Это объясняется тем, что в `mixed`-сценарии присутствуют и относительно дешевые вставки, и более дорогие удаления. Операция `pop` обычно тяжелее `push`, потому что `sift down`

из корня часто проходит больше уровней, чем `sift up` при случайной вставке.

Теоретически каждая операция `push` или `pop` имеет сложность $O(\log N)$, поэтому серия из N операций имеет оценку $O(N \log N)$. При этом реальное время зависит от текущего размера кучи и от соотношения вставок и удалений.

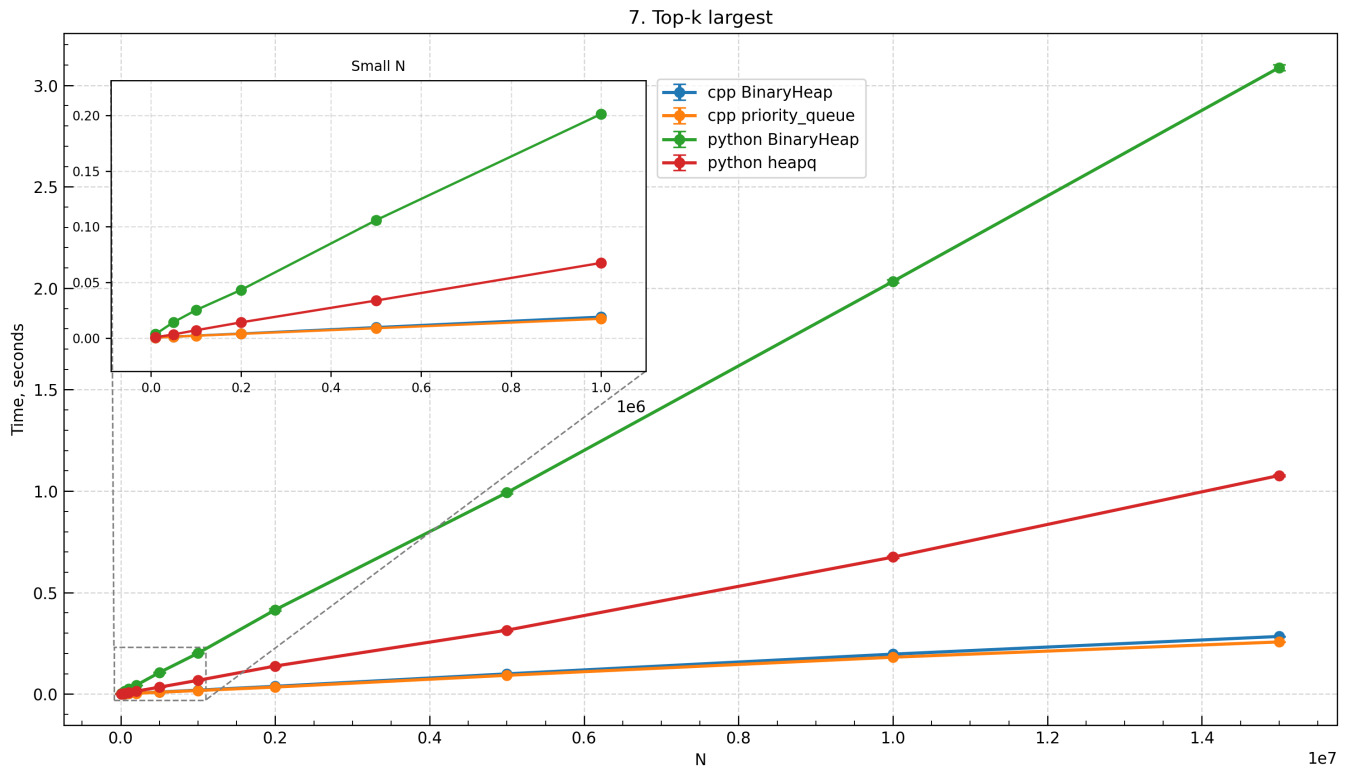
Собственная Python-реализация снова оказывается самой медленной, поскольку обе основные операции требуют циклов и перестановок на уровне Python-кода. Особенно дорогими становятся удаления, где нужно выполнять `sift down` и постоянно сравнивать текущий элемент с детьми.

`heapq` и C++-реализации справляются с этой нагрузкой значительно лучше. Среди C++-реализаций собственная `BinaryHeap` остается быстрее `std::priority_queue`, что говорит о малых постоянных расходах собственной специализированной реализации.

Таким образом, benchmark-case `Alternating push/pop` показывает поведение кучи в динамическом сценарии. Он хорошо демонстрирует, что производительность зависит не только от асимптотики отдельных операций, но и от их сочетания, текущего размера структуры и постоянных расходов реализации.

Top-k largest

В данном benchmark-case измеряется использование кучи для поиска k наибольших элементов. В эксперименте используется фиксированное значение $k = 100$. Алгоритм поддерживает min-heap размера не больше k : в куче хранятся текущие k лучших элементов, а минимальный среди них находится в корне.



note

1 7. Top-k largest

2

3	N	cpp_BinaryHeap	cpp_priority_queue	python_BinaryHeap	heapq
4					
5	1000000	~0.019 s	~0.017 s	~0.199 s	~0.068 s
6	2000000	~0.038 s	~0.034 s	~0.411 s	~0.138 s
7	5000000	~0.099 s	~0.092 s	~0.996 s	~0.317 s
8	10000000	~0.197 s	~0.181 s	~2.034 s	~0.674 s
9	15000000	~0.284 s	~0.255 s	~3.085 s	~1.075 s

Данный benchmark-case отличается от остальных тем, что размер кучи не растёт до N . Он ограничен значением $k = 100$. Поэтому каждая операция изменения кучи имеет сложность $O(\log k)$, а не $O(\log N)$.

Алгоритм работает следующим образом: первые k элементов добавляются в кучу, а затем каждый новый элемент сравнивается с минимумом среди текущих top-k. Если новый элемент меньше или равен корню, он не может попасть в ответ, и операция завершается после одного сравнения. Если он больше корня, минимальный элемент удаляется, а новый вставляется в кучу.

Так как `k` фиксировано и мало по сравнению с `N`, общая сложность алгоритма равна $O(N \log k)$, что при фиксированном `k` практически ведет себя как линейная зависимость от `N`. Это хорошо видно на графике.

Времена в этом кейсе значительно меньше, чем в сценариях массового удаления или чередования `push/pop`. Причина в том, что куча всегда маленькая, а большая часть элементов требует только сравнения с корнем без изменения структуры.

Собственная Python-реализация остается медленнее `heapq`, но разрыв меньше, чем в тяжелых сценариях с большой кучей. Это связано с тем, что `sift up` и `sift down` выполняются на куче размера всего `100`, то есть имеют очень маленькую глубину.

Среди C++-реализаций `std::priority_queue` в этом кейсе немного быстрее собственной `BinaryHeap`. Разница небольшая и связана с постоянными факторами: при маленьком размере кучи основная стоимость приходится не на глубину дерева, а на детали реализации отдельных операций, сравнение с корнем и служебную логику цикла.

Таким образом, benchmark-case `Top-k largest` показывает один из наиболее практичных сценариев использования кучи. Он демонстрирует, что для поиска небольшого числа лучших элементов не нужно сортировать весь массив: достаточно поддерживать кучу фиксированного размера, что дает почти линейное время по числу входных элементов.

Общий вывод по benchmark-ам Binary Heap

По результатам benchmark-ов видно, что собственные реализации корректно воспроизводят основные свойства бинарной кучи. В простых сценариях графики соответствуют ожидаемым теоретическим оценкам: `heapify` ведет себя линейно, массовые удаления и смешанные операции дают более тяжелый рост, а вставка сильно зависит от порядка входных данных.

Наиболее важное наблюдение связано с операцией `push`. Хотя в худшем случае она имеет сложность $O(\log N)$, на практике ее стоимость сильно зависит от того, сколько уровней проходит новый элемент при `sift up`. При возрастающей последовательности почти все вставки завершаются сразу, поэтому график близок к линейному. При убывающей последовательности каждый новый элемент поднимается почти до корня, и время резко возрастает. Случайная вставка занимает промежуточное положение.

Операция `heapify` подтверждает преимущество bottom-up построения кучи. Если все элементы известны заранее, построение кучи за $O(N)$ оказывается значительно выгоднее, чем последовательные вставки в неблагоприятном порядке.

Самыми тяжелыми сценариями оказываются `Pop all after heapify` и `Alternating push/pop`. В них активно используется `sift down`, который часто проходит значительную часть высоты дерева. Поэтому эти кейсы лучше показывают реальную стоимость удаления из кучи и работу структуры в динамических сценариях.

Собственная Python-реализация во всех benchmark-ах заметно уступает `heapq`. Это ожидаемо: даже при той же алгоритмической идее операции собственной кучи выполняются на уровне Python-кода, где каждый цикл, вызов метода, сравнение, вычисление индекса и перестановка имеют существенные накладные расходы.

C++-реализации во многих сценариях оказываются наиболее быстрыми. При этом собственная `BinaryHeap` часто быстрее `std::priority_queue`, особенно в простых и регулярных сценариях. Это можно объяснить тем, что собственная реализация специализирована под конкретный вариант кучи и имеет минимальную внутреннюю логику, тогда как `priority_queue` является более универсальным контейнером-адаптером.

Однако библиотечные реализации, особенно `heapq`, показывают, насколько сильно практическая производительность зависит от качества реализации. Они сохраняют ту же асимптотику, но имеют меньшие постоянные расходы и лучше подходят для типичного использования.

Главный вывод состоит в том, что для `binary heap` важна не только асимптотика операций, но и конкретный характер входных данных. Одна и та же операция `push` может вести себя почти как $O(1)$ на благоприятном порядке и как $O(\log N)$ на неблагоприятном. Поэтому benchmark-и по разным сценариям хорошо показывают, как теоретические свойства структуры данных проявляются на практике.